

Lecture 10

Object-Oriented Design Patterns

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Object-Oriented Design Patterns

- All 23 OO *design patterns* from Gang of Four book (Design Patterns: Elements of Reusable Object-Oriented Software (1994))

Object-Oriented Design Patterns

<u>Creational</u>	<u>Structural</u>	<u>Behavioral</u>
Abstract Factory*	Adapter	Chain of Responsibility
Builder	Bridge	Command*
Factory Method*	Composite	Iterator
Prototype	Decorator	Interpreter
Singleton*	Facade*	Mediator
	Flyweight*	Memento
	Proxy*	Observer

Creational

Structural

Behavioral

State

Strategy

Template Method

Visitor

Motivating Example 1

Evolution of Trust example from earlier lecture

- Strategy interface implementers
 - AlwaysCooperateStrategy
 - AlwaysCheatStrategy
 - CopycatStrategy
 - DetectiveStrategy
 - GrudgerStrategy
- Player contains Strategy

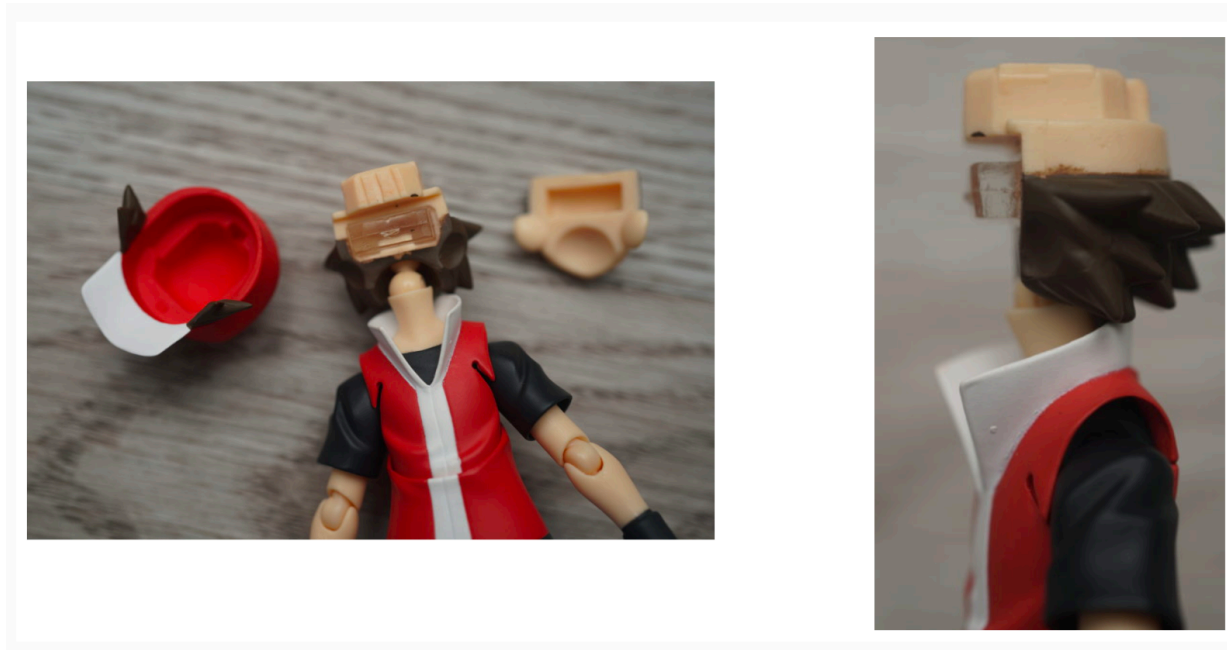
Motivating Example 1

Evolution of Trust example from earlier lecture

- Varying aspect was extracted into separate hierarchy

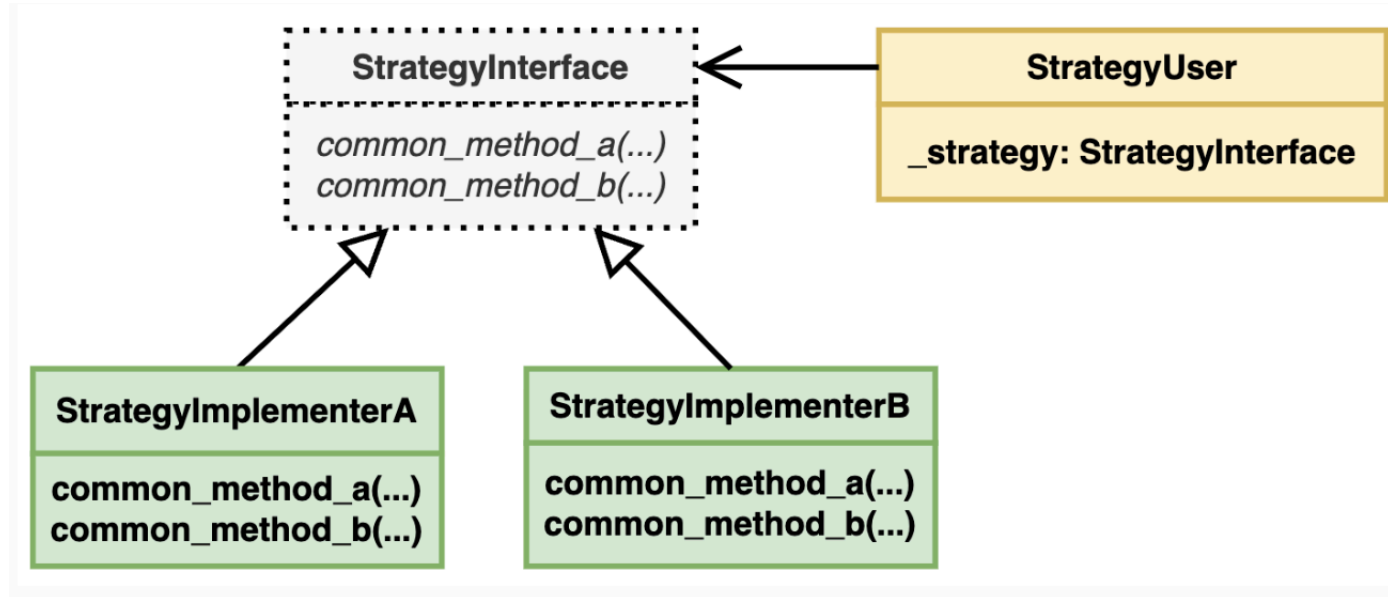
Motivating Example 1

- Source: <http://justveryrandom.com/toy-review-figma-red/>



Strategy Pattern

Strategy pattern: Allows interchangeable behaviors via composition



Strategy Pattern

Part

Notes

Strategy interface

Same method signature for behavior variations

Strategy implementers

One class per behavior variant

Strategy user

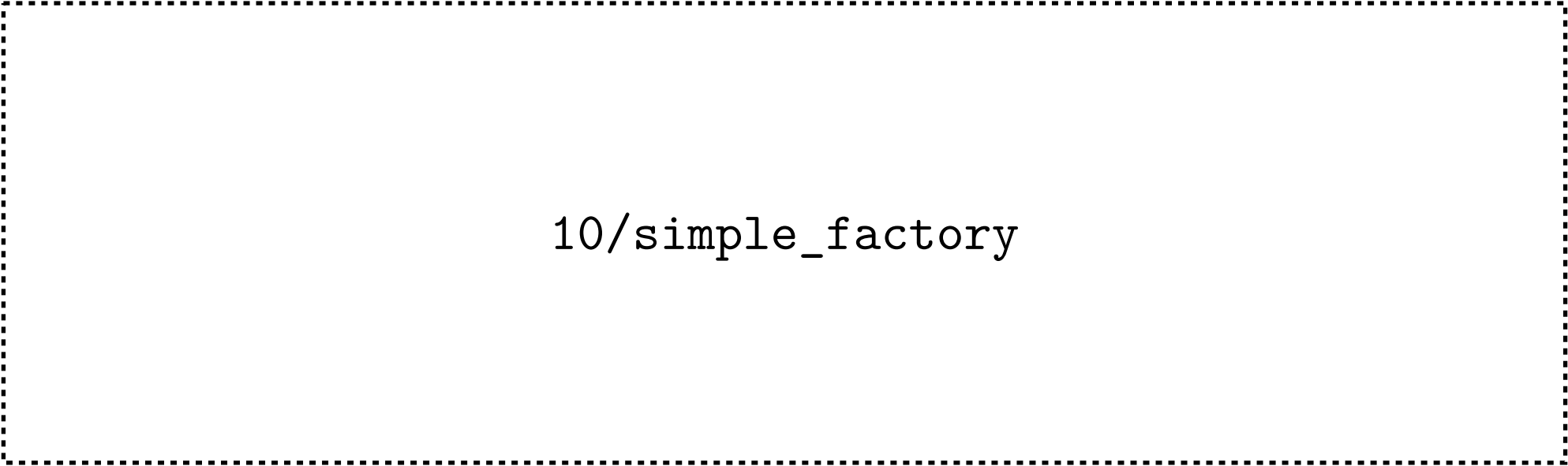
Contains common aspects

Strategy Pattern

10/strategy_pattern

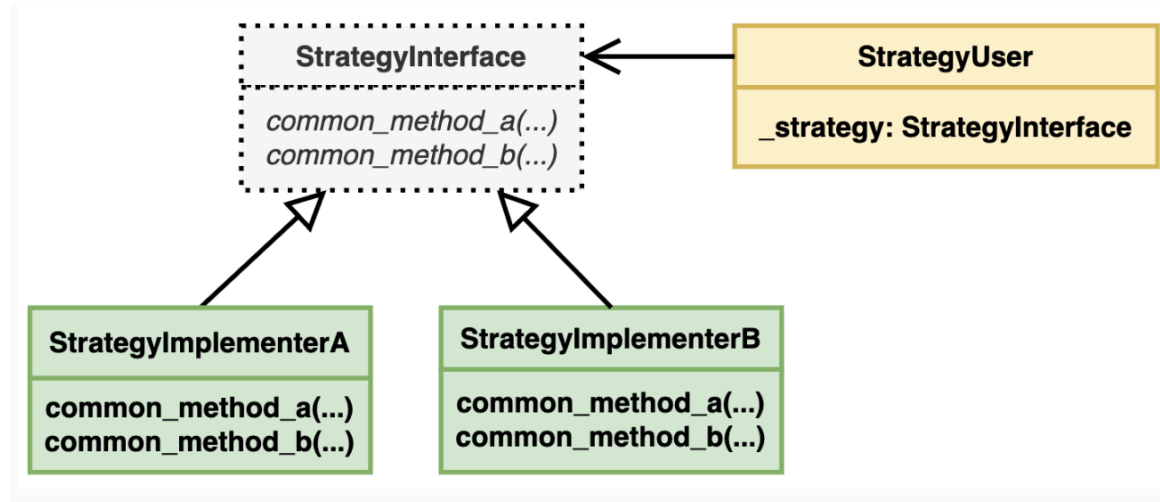
Aside: Simple Factory Pattern

Simple factory pattern: Delegates responsibility of object instantiation to a factory class (not part of GoF design patterns)



10/simple_factory

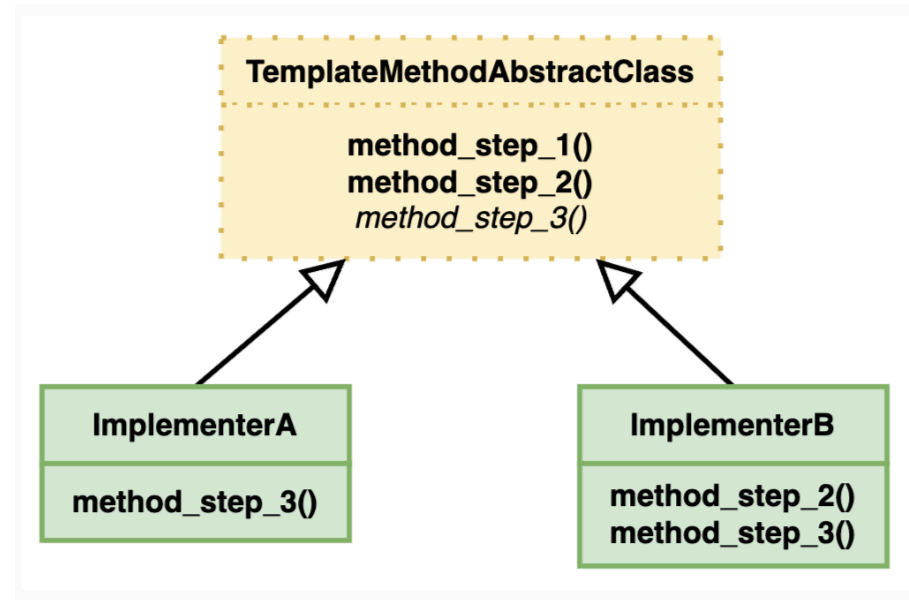
Motivating Example 2



Strategy pattern uses composition; how do we use inheritance instead?

Template Method Pattern

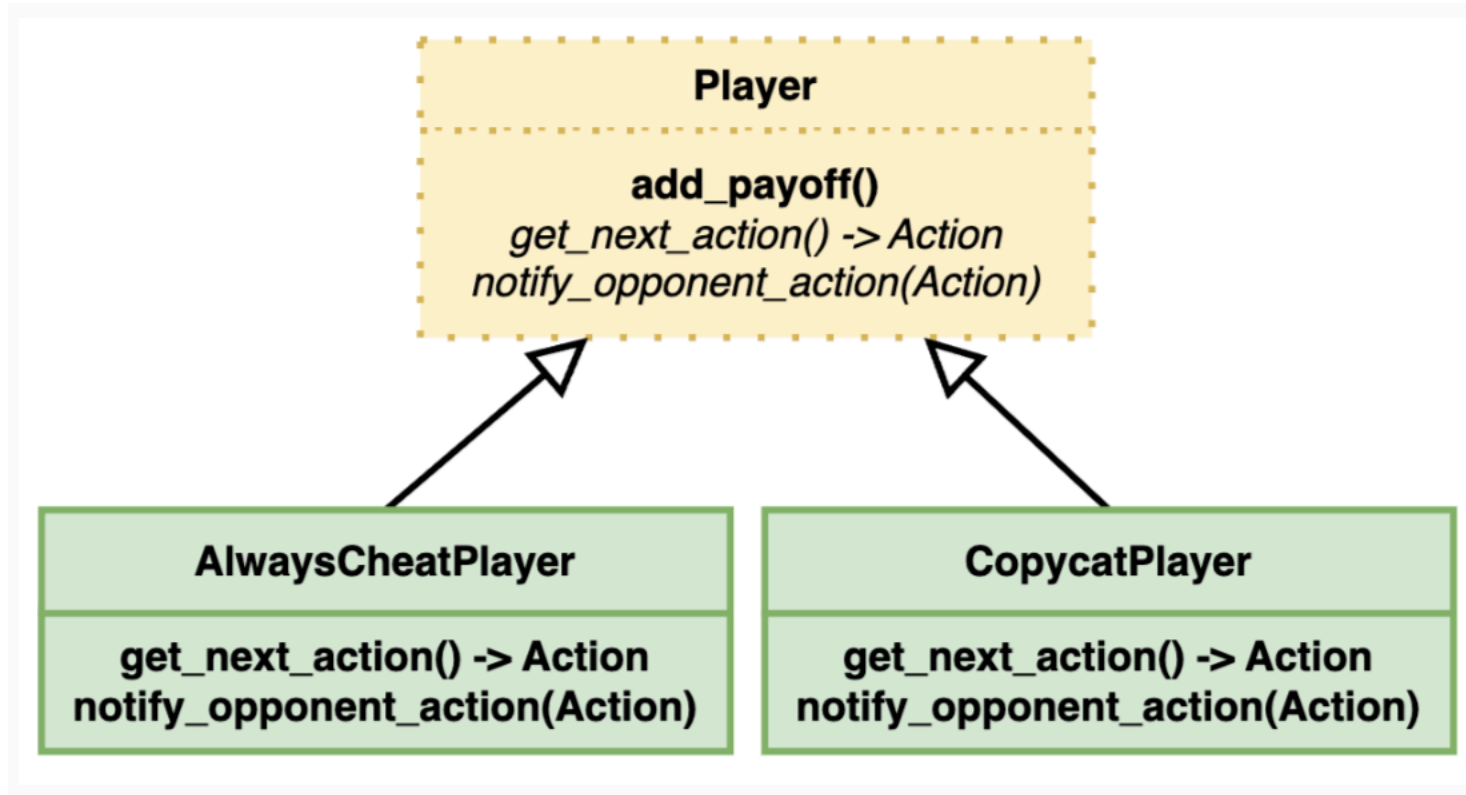
Template method pattern: Delegates implementation of subset of steps to subclasses



Template Method Pattern

<i>Part</i>	<i>Notes</i>
Abstract class	Exposes methods allowed to be implemented or overridden
Implementers	Implements abstract methods, may override public concrete methods

Template Method Pattern



Template Method Pattern

10/template_method

Template Method Pattern

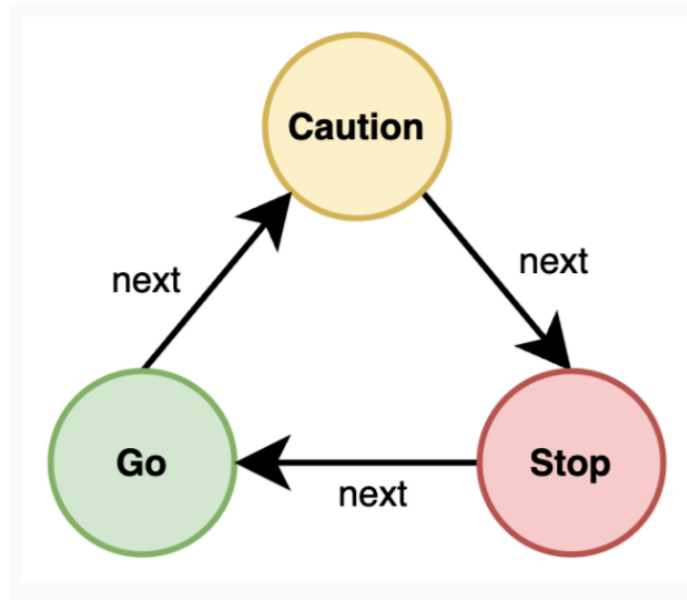
Insight: Superclass methods can serve as “hooks” for subclasses to inject or alter behaviors via overriding

- <https://www.scrapy.org/>
- https://docs.scrapy.org/en/latest/_modules/scrapy/spiders.html#Spider

10/template_method_scrapy

Motivating Example 3

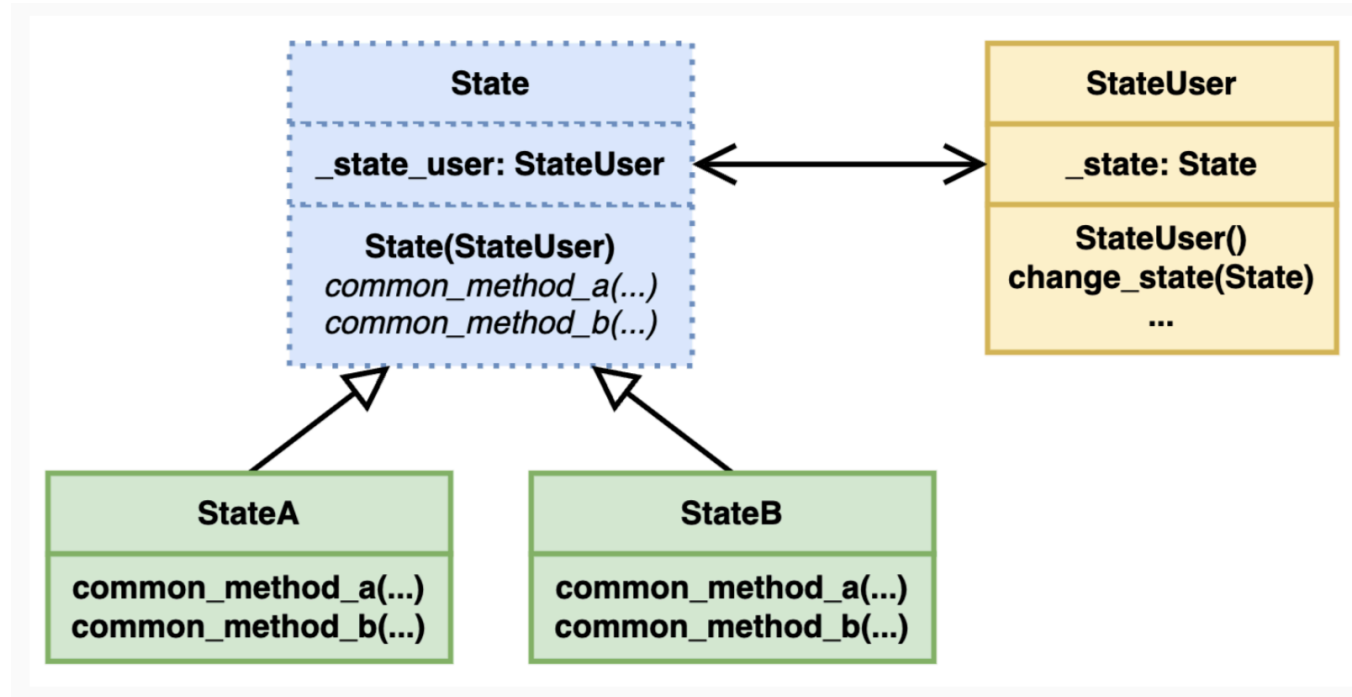
Finite state machine (FSM) / Deterministic finite automaton (DFA) for a traffic light (no blinking green)



Motivating Example 3

10/conditionals

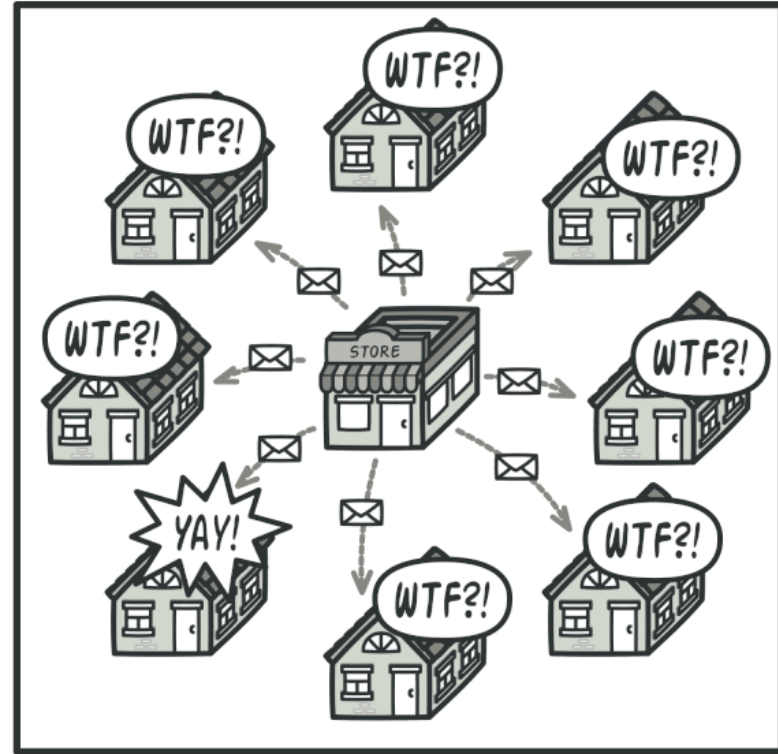
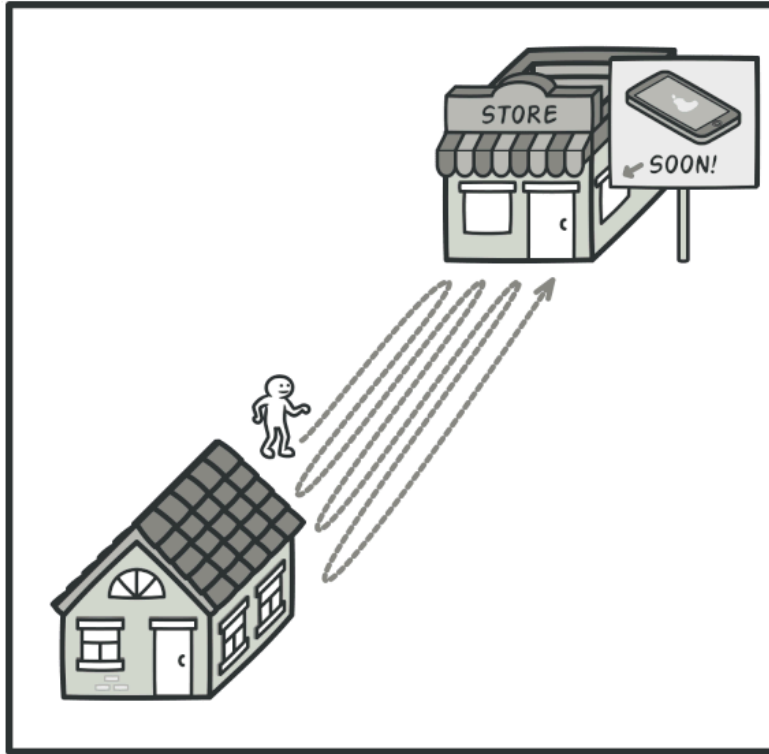
State Pattern



State Pattern

10/state_pattern

Motivating Example 4



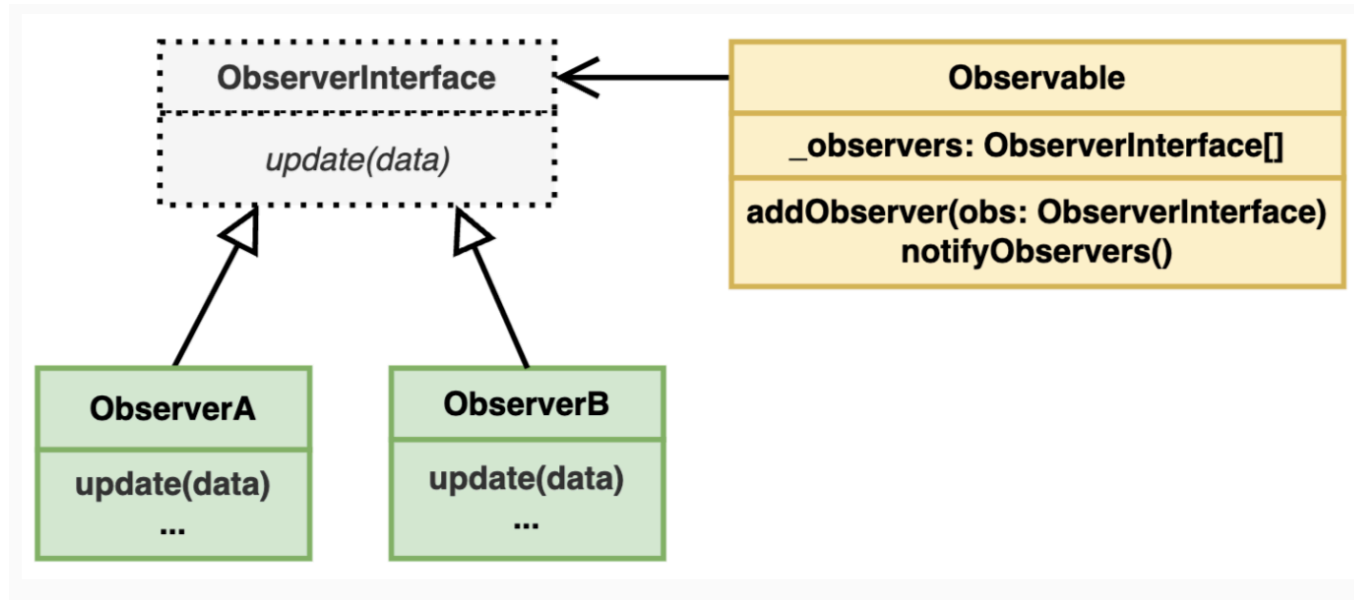
Motivating Example 4

Imagine that you have two types of objects: a `Customer` and a `Store`. The customer is very interested in a particular brand of product (say, it's a new model of the iPhone) which should become available in the store very soon.

The customer could visit the store every day and check product availability. But while the product is still en route, most of these trips would be pointless.

Observer Pattern

Observer pattern: Allows observable to notify observers without knowing who they are



Observer Pattern

Observer pattern: Allows observable to notify observers without knowing who they are

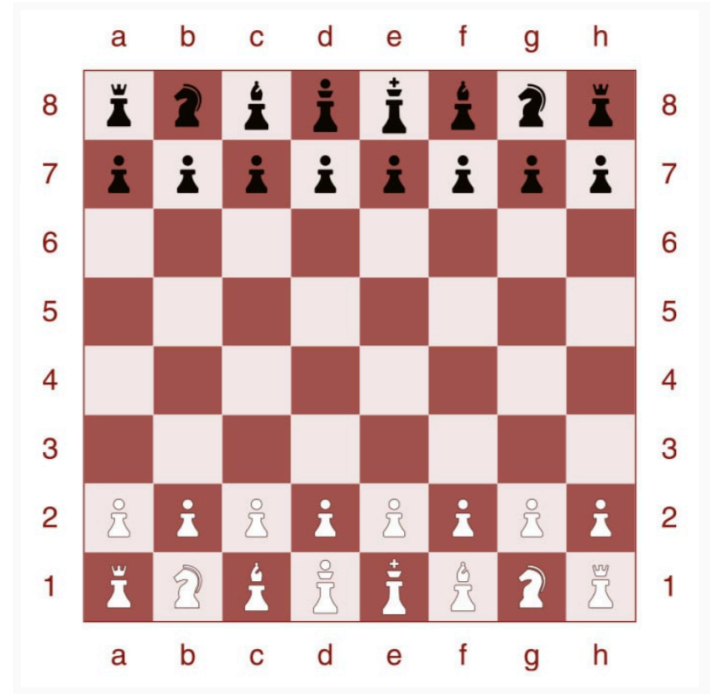
- Observer = Subscriber = Listener
- Observable = Publisher

Observer Pattern

10/observer_pattern

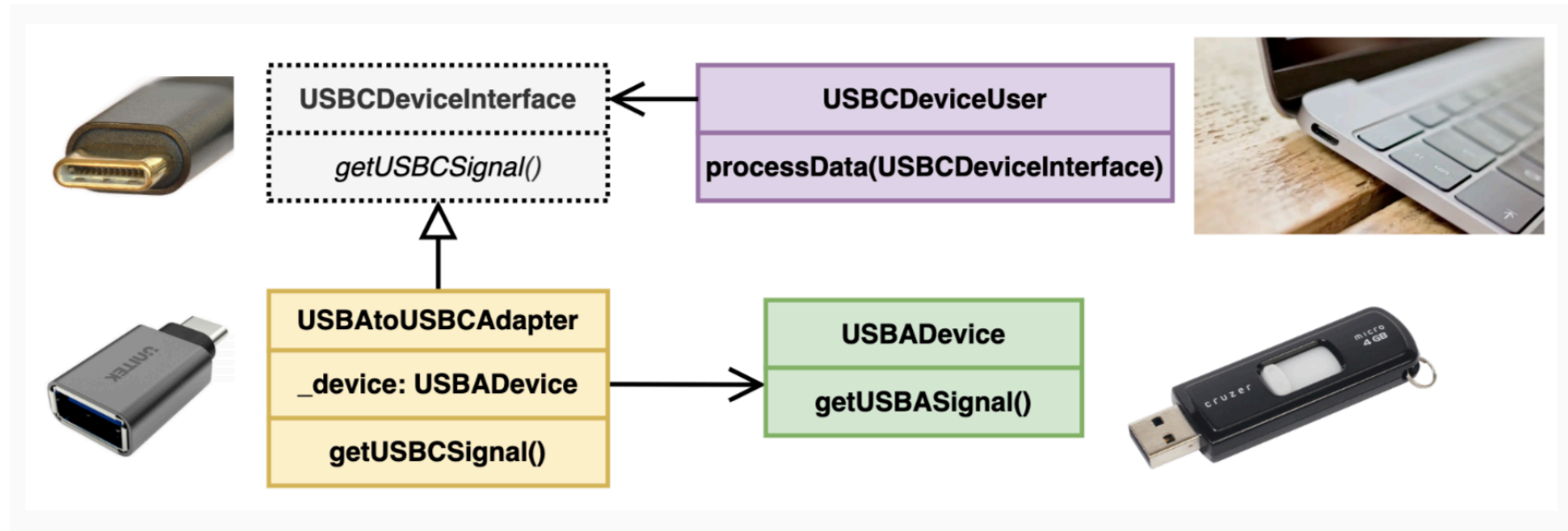
Motivating Example 5

10/chess



Adapter Pattern

Adapter pattern: Converts an incompatible interface to a compatible one



<i>Part</i>	<i>Relation to other parts</i>	<i>Notes</i>
Incompatible interface	Contained in adapter	Already exists, can be object
Compatible interface	Implemented in adapter	Interface expected by client
Adapter class	Implements compatible interface, contains incompatible interface	Contains conversion logic

Adapter Pattern

10/adapter_pattern

References

- <https://refactoring.guru/design-patterns>
- <https://gameprogrammingpatterns.com/contents.html>
- Head First Design Patterns 2e (Freeman, etc.)